

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import sys
print(sys.executable)
```

C:\Users\USER\miniconda3\envs\myenv\python.exe

Load the two datasets

```
In [2]: import sys
!{sys.executable} -m pip install openpyxl
```

Requirement already satisfied: openpyxl in C:\Users\USER\miniconda3\envs\myenv\Lib\site-packages (3.1.5)

Requirement already satisfied: et-xmlfile in C:\Users\USER\miniconda3\envs\myenv\Lib\site-packages (from openpyxl) (2.0.0)

```
In [3]: transactions = pd.read_excel('QVI_transaction_data.xlsx')
customers = pd.read_csv('QVI_purchase_behaviour.csv')
```

```
In [4]: transactions.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 264836 entries, 0 to 264835
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0   DATE                  264836 non-null int64
1   STORE_NBR             264836 non-null int64
2   LYLTY_CARD_NBR        264836 non-null int64
3   TXN_ID                264836 non-null int64
4   PROD_NBR              264836 non-null int64
5   PROD_NAME             264836 non-null str
6   PROD_QTY              264836 non-null int64
7   TOT_SALES             264836 non-null float64
dtypes: float64(1), int64(6), str(1)
memory usage: 16.2 MB
```

```
In [5]: transactions.describe()
```

Out[5]:

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PR
count	264836.000000	264836.00000	2.648360e+05	2.648360e+05	264836.000000	26483
mean	43464.036260	135.08011	1.355495e+05	1.351583e+05	56.583157	
std	105.389282	76.78418	8.057998e+04	7.813303e+04	32.826638	
min	43282.000000	1.00000	1.000000e+03	1.000000e+00	1.000000	
25%	43373.000000	70.00000	7.002100e+04	6.760150e+04	28.000000	
50%	43464.000000	130.00000	1.303575e+05	1.351375e+05	56.000000	
75%	43555.000000	203.00000	2.030942e+05	2.027012e+05	85.000000	
max	43646.000000	272.00000	2.373711e+06	2.415841e+06	114.000000	20

In [6]:

```
print(transactions.shape)
transactions.info()
```

(264836, 8)
<class 'pandas.DataFrame'>
RangeIndex: 264836 entries, 0 to 264835
Data columns (total 8 columns):
Column Non-Null Count Dtype
--- -
0 DATE 264836 non-null int64
1 STORE_NBR 264836 non-null int64
2 LYLTY_CARD_NBR 264836 non-null int64
3 TXN_ID 264836 non-null int64
4 PROD_NBR 264836 non-null int64
5 PROD_NAME 264836 non-null str
6 PROD_QTY 264836 non-null int64
7 TOT_SALES 264836 non-null float64
dtypes: float64(1), int64(6), str(1)
memory usage: 16.2 MB

In [9]:

```
# identifying data anomalies and flagged outlier
transactions[transactions['PROD_QTY'] == 200]
```

Out[9]:

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_NAME	PROD_Q
69762	43331	226	226000	226201	4	Dorito Corn Chp Supreme 380g	20
69763	43605	226	226000	226210	4	Dorito Corn Chp Supreme 380g	20

```
In [8]: # identifying data anomalies and flagged outlier
transactions[transactions['LYLTY_CARD_NBR'] == 226000]
```

```
Out[8]:
```

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_NAME	PROD_QTY
69762	43331	226	226000	226201	4	Dorito Corn Chp Supreme 380g	20
69763	43605	226	226000	226210	4	Dorito Corn Chp Supreme 380g	20

```
In [10]: #removing commercial transactions
transactions = transactions[transactions['LYLTY_CARD_NBR'] != 226000]

# Confirm it's gone
print(transactions[transactions['LYLTY_CARD_NBR'] == 226000])
print(transactions.shape)
```

Empty DataFrame

Columns: [DATE, STORE_NBR, LYLTY_CARD_NBR, TXN_ID, PROD_NBR, PROD_NAME, PROD_QTY, TO T_SALES]

Index: []

(264834, 8)

```
In [11]: #covert excel serial number to actual date
transactions['DATE'] = pd.to_datetime(transactions['DATE'], origin='1899-12-30', un
transactions['DATE'].head())
```

```
Out[11]: 0    2018-10-17
1    2019-05-14
2    2019-05-20
3    2018-08-17
4    2018-08-18
Name: DATE, dtype: datetime64[s]
```

```
In [12]: print(transactions['DATE'].min())
print(transactions['DATE'].max())
print((transactions['DATE'].max() - transactions['DATE'].min()).days + 1)
print(transactions['DATE'].nunique())
```

2018-07-01 00:00:00

2019-06-30 00:00:00

365

364

```
In [13]: # identifying unique date
all_dates = pd.date_range(start=transactions['DATE'].min(), end=transactions['DATE']
missing_dates = all_dates[~all_dates.isin(transactions['DATE'])]
missing_dates
```

```
Out[13]: DatetimeIndex(['2018-12-25'], dtype='datetime64[s]', freq='D')
```

```
In [14]: #checking product name
pd.set_option('display.max_rows', None)
transactions['PROD_NAME'].unique()
```

```
Out[14]: <StringArray>
[ 'Natural Chip          Compny SeaSalt175g',
  'CCs Nacho Cheese      175g',
  'Smiths Crinkle Cut   Chips Chicken 170g',
  'Smiths Chip Thinly   S/Cream&Onion 175g',
  'Kettle Tortilla ChpsHny&Jlpno Chili 150g',
  'Old El Paso Salsa    Dip Tomato Mild 300g',
  'Smiths Crinkle Chips Salt & Vinegar 330g',
  'Grain Waves          Sweet Chilli 210g',
  'Doritos Corn Chip Mexican Jalapeno 150g',
  'Grain Waves Sour     Cream&Chives 210G',
  ...
  'Doritos Cheese       Supreme 330g',
  'Smiths Crinkle Cut   Snag&Sauce 150g',
  'WW Sour Cream &OnionStacked Chips 160g',
  'RRD Lime & Pepper    165g',
  'Natural ChipCo Sea   Salt & Vinegr 175g',
  'Red Rock Deli Chikn&Garlic Aioli 150g',
  'RRD SR Slow Rst      Pork Belly 150g',
  'RRD Pc Sea Salt      165g',
  'Smith Crinkle Cut    Bolognese 150g',
  'Doritos Salsa Mild   300g']
Length: 114, dtype: str
```

```
In [15]: transactions[transactions['PROD_NAME'].str.contains('salsa|dip', case=False)][ 'PROD
```

```
Out[15]: <StringArray>
['Old El Paso Salsa    Dip Tomato Mild 300g',
 'Red Rock Deli SR     Salsa & Mzzrilla 150g',
 'Smiths Crinkle Cut   French OnionDip 150g',
 'Smiths Crinkle Cut   Tomato Salsa 150g',
 'Doritos Salsa        Medium 300g',
 'Old El Paso Salsa    Dip Chnky Tom Ht300g',
 'Woolworths Mild      Salsa 300g',
 'Old El Paso Salsa    Dip Tomato Med 300g',
 'Woolworths Medium    Salsa 300g',
 'Doritos Salsa Mild   300g']
Length: 10, dtype: str
```

```
In [16]: transactions[transactions['PROD_NAME'].str.contains('French OnionDip', case=False)]
```

```
Out[16]: <StringArray>
['Smiths Crinkle Cut   French OnionDip 150g']
Length: 1, dtype: str
```

**remove any product where the name contains
"salsa" (that catches all 6 genuine dip/salsa jars),**

and keep everything else, including the French OnionDip chips.

```
In [17]: transactions = transactions[~transactions['PROD_NAME'].str.contains('salsa', case=F

# Confirm removal
print(transactions[transactions['PROD_NAME'].str.contains('salsa', case=False)])
print(transactions.shape)
print(transactions['PROD_NAME'].nunique())
```

Empty DataFrame

Columns: [DATE, STORE_NBR, LYLTY_CARD_NBR, TXN_ID, PROD_NBR, PROD_NAME, PROD_QTY, TO
T_SALES]

Index: []

(246740, 8)

105

```
In [19]: transactions[transactions['PROD_NAME'].str.extract(r'(\d+)g')[0].isna()][ 'PROD_NAME
```

```
Out[19]: <StringArray>
[   'Grain Waves Sour    Cream&Chives 210G',
   'Red Rock Deli Sp    Salt & Truffle 150G',
   'Smiths Thinly        Swt Chli&S/Cream175G']
Length: 3, dtype: str
```

```
In [20]: transactions['PACK_SIZE'] = transactions['PROD_NAME'].str.extract(r'(\d+)[gG]')[0].
transactions['PACK_SIZE'].describe()
```

```
Out[20]: count    246740.000000
mean         175.583521
std           59.432118
min           70.000000
25%          150.000000
50%          170.000000
75%          175.000000
max          380.000000
Name: PACK_SIZE, dtype: float64
```

```
In [21]: transactions['BRAND'] = transactions['PROD_NAME'].str.split().str[0]
transactions['BRAND'].value_counts()
```

```
Out[21]: BRAND
Kettle      41288
Smiths      27390
Pringles    25102
Doritos     22041
Thins       14075
RRD         11894
Infuzions   11057
WW          10320
Cobs        9693
Tostitos    9471
Twisties    9454
Tyrrells    6442
Grain       6272
Natural     6050
Cheezels    4603
CCs         4551
Red         4427
Dorito      3183
Infzns      3144
Smith       2963
Cheetos     2927
Snbts       1576
Burger      1564
Woolworths  1516
GrnWves     1468
Sunbites    1432
NCC         1419
French      1418
Name: count, dtype: int64
```

```
In [22]: transactions[transactions['BRAND'].isin(['French', 'Burger'])]['PROD_NAME'].unique()
```

```
Out[22]: <StringArray>
['Burger Rings 220g', 'French Fries Potato Chips 175g']
Length: 2, dtype: str
```

fixes for the real duplicates

```
In [23]: #fixes for the real duplicates
brand_fixes = {
    'Dorito': 'Doritos',
    'Smith': 'Smiths',
    'Infzns': 'Infuzions',
    'Grain': 'Grain Waves',
    'GrnWves': 'Grain Waves',
    'RRD': 'Red Rock Deli',
    'Red': 'Red Rock Deli',
    'WW': 'Woolworths',
    'Snbts': 'Sunbites',
    'NCC': 'Natural Chip Co'
}
```

```
transactions['BRAND'] = transactions['BRAND'].replace(brand_fixes)
transactions['BRAND'].value_counts()
```

```
Out[23]: BRAND
Kettle          41288
Smiths          30353
Doritos         25224
Pringles        25102
Red Rock Deli   16321
Infuzions       14201
Thins           14075
Woolworths      11836
Cobs            9693
Tostitos        9471
Twisties        9454
Grain Waves     7740
Tyrrells        6442
Natural         6050
Cheezels        4603
CCs             4551
Sunbites        3008
Cheetos         2927
Burger          1564
Natural Chip Co 1419
French          1418
Name: count, dtype: int64
```

```
In [24]: transactions[transactions['BRAND'].isin(['Natural', 'Natural Chip Co'])['PROD_NAME
```

```
Out[24]: <StringArray>
[ 'Natural Chip      Compny SeaSalt175g',
  'NCC Sour Cream &  Garden Chives 175g',
  'Natural ChipCo    Hony Soy Chckn175g',
  'Natural Chip Co   Tmato Hrb&Spce 175g',
  'Natural ChipCo Sea Salt & Vinegr 175g']
Length: 5, dtype: str
```

```
In [25]: transactions['BRAND'] = transactions['BRAND'].replace({'Natural': 'Natural Chip Co'
transactions['BRAND'].value_counts()
```

```
Out[25]: BRAND
Kettle          41288
Smiths          30353
Doritos         25224
Pringles        25102
Red Rock Deli   16321
Infuzions       14201
Thins           14075
Woolworths      11836
Cobs            9693
Tostitos        9471
Twisties        9454
Grain Waves     7740
Natural Chip Co 7469
Tyrrells        6442
Cheezels        4603
CCs             4551
Sunbites        3008
Cheetos         2927
Burger          1564
French          1418
Name: count, dtype: int64
```

```
In [26]: transactions[['STORE_NBR', 'LYLTY_CARD_NBR', 'TXN_ID']].describe()
```

```
Out[26]:
```

	STORE_NBR	LYLTY_CARD_NBR	TXN_ID
count	246740.000000	2.467400e+05	2.467400e+05
mean	135.050361	1.355303e+05	1.351304e+05
std	76.786971	8.071520e+04	7.814760e+04
min	1.000000	1.000000e+03	1.000000e+00
25%	70.000000	7.001500e+04	6.756875e+04
50%	130.000000	1.303670e+05	1.351815e+05
75%	203.000000	2.030832e+05	2.026522e+05
max	272.000000	2.373711e+06	2.415841e+06

```
In [27]: transactions.to_csv('QVI_transaction_data_clean.csv', index=False)
print("Saved!")
transactions.shape
```

Saved!

```
Out[27]: (246740, 10)
```

```
In [28]: customers.info()
customers.describe(include='all')
```



```
<class 'pandas.DataFrame'>
RangeIndex: 72637 entries, 0 to 72636
Data columns (total 3 columns):
#   Column                Non-Null Count  Dtype
---  -
0   LYLTY_CARD_NBR        72637 non-null  int64
1   LIFESTAGE              72637 non-null  str
2   PREMIUM_CUSTOMER      72637 non-null  str
dtypes: int64(1), str(2)
memory usage: 1.7 MB
```

Out[28]:

	LYLTY_CARD_NBR	LIFESTAGE	PREMIUM_CUSTOMER
count	7.263700e+04	72637	72637
unique	NaN	7	3
top	NaN	RETIREEES	Mainstream
freq	NaN	14805	29245
mean	1.361859e+05	NaN	NaN
std	8.989293e+04	NaN	NaN
min	1.000000e+03	NaN	NaN
25%	6.620200e+04	NaN	NaN
50%	1.340400e+05	NaN	NaN
75%	2.033750e+05	NaN	NaN
max	2.373711e+06	NaN	NaN

In [29]: `customers['LYLTY_CARD_NBR'].duplicated().sum()`

Out[29]: `np.int64(0)`

Now let's join the two datasets.

In [30]:

```
merged_data = transactions.merge(customers, on='LYLTY_CARD_NBR', how='left')
merged_data.head()
print(merged_data.shape)
merged_data.isnull().sum()
```

(246740, 12)

```
Out[30]: DATE          0
        STORE_NBR      0
        LYLTY_CARD_NBR 0
        TXN_ID         0
        PROD_NBR       0
        PROD_NAME      0
        PROD_QTY       0
        TOT_SALES      0
        PACK_SIZE      0
        BRAND          0
        LIFESTAGE      0
        PREMIUM_CUSTOMER 0
        dtype: int64
```

```
In [31]: merged_data.to_csv('QVI_merged_data.csv', index=False)
        print("Saved!")
        merged_data.shape
```

Saved!

```
Out[31]: (246740, 12)
```

Data analysis and customer segments

```
In [32]: sales_by_segment = merged_data.groupby(['LIFESTAGE', 'PREMIUM_CUSTOMER'])['TOT_SALE
        sales_by_segment = sales_by_segment.sort_values('TOT_SALES', ascending=False)
        sales_by_segment
```

Out[32]:

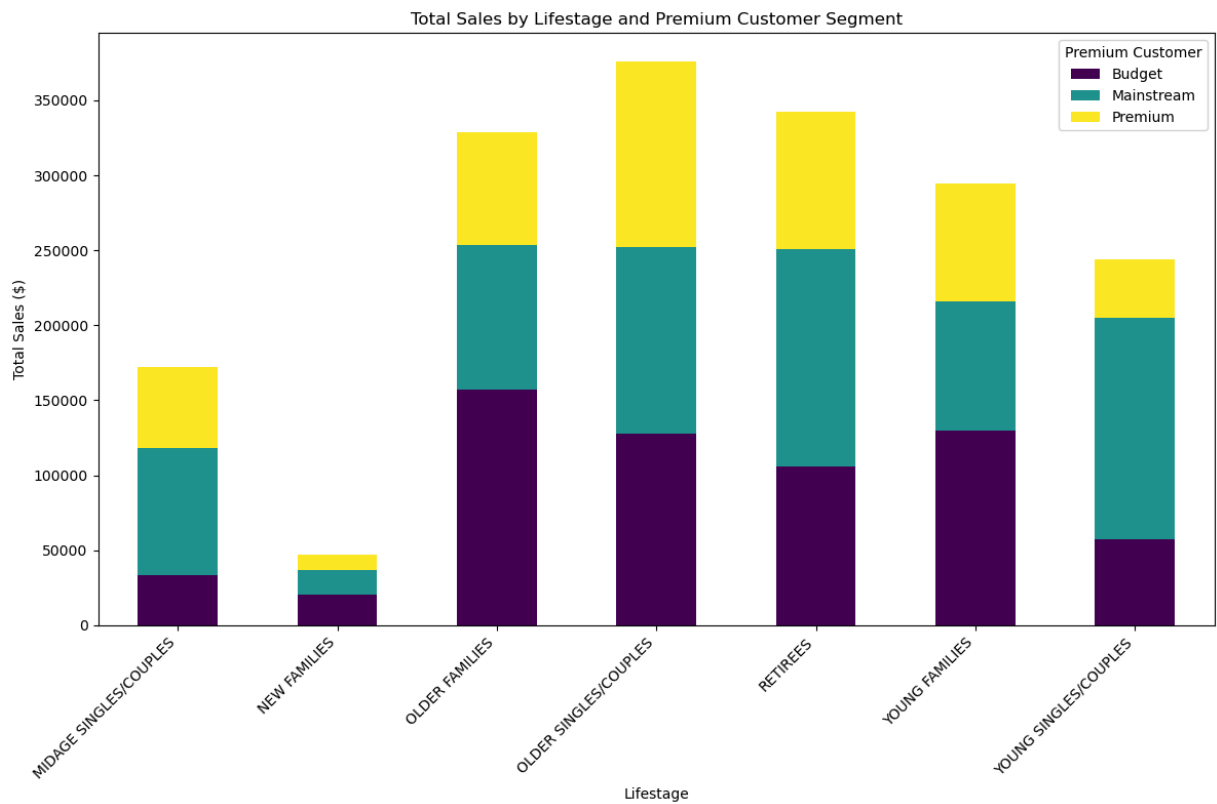
	LIFESTAGE	PREMIUM_CUSTOMER	TOT_SALES
6	OLDER FAMILIES	Budget	156863.75
19	YOUNG SINGLES/COUPLES	Mainstream	147582.20
13	RETIREEES	Mainstream	145168.95
15	YOUNG FAMILIES	Budget	129717.95
9	OLDER SINGLES/COUPLES	Budget	127833.60
10	OLDER SINGLES/COUPLES	Mainstream	124648.50
11	OLDER SINGLES/COUPLES	Premium	123537.55
12	RETIREEES	Budget	105916.30
7	OLDER FAMILIES	Mainstream	96413.55
14	RETIREEES	Premium	91296.65
16	YOUNG FAMILIES	Mainstream	86338.25
1	MIDAGE SINGLES/COUPLES	Mainstream	84734.25
17	YOUNG FAMILIES	Premium	78571.70
8	OLDER FAMILIES	Premium	75242.60
18	YOUNG SINGLES/COUPLES	Budget	57122.10
2	MIDAGE SINGLES/COUPLES	Premium	54443.85
20	YOUNG SINGLES/COUPLES	Premium	39052.30
0	MIDAGE SINGLES/COUPLES	Budget	33345.70
3	NEW FAMILIES	Budget	20607.45
4	NEW FAMILIES	Mainstream	15979.70
5	NEW FAMILIES	Premium	10760.80

```

In [33]: plt.figure(figsize=(12, 8))
pivot = merged_data.groupby(['LIFESTAGE', 'PREMIUM_CUSTOMER'])['TOT_SALES'].sum().u
pivot.plot(kind='bar', stacked=True, figsize=(12, 8), colormap='viridis')
plt.title('Total Sales by Lifestage and Premium Customer Segment')
plt.xlabel('Lifestage')
plt.ylabel('Total Sales ($)')
plt.xticks(rotation=45, ha='right')
plt.legend(title='Premium Customer')
plt.tight_layout()
plt.savefig('sales_by_segment.png', dpi=150)
plt.show()

```

<Figure size 1200x800 with 0 Axes>



```
In [34]: customer_counts = merged_data.groupby(['LIFESTAGE', 'PREMIUM_CUSTOMER'])['LYLTY_CARD']
customer_counts.columns = ['LIFESTAGE', 'PREMIUM_CUSTOMER', 'CUSTOMER_COUNT']

segment_summary = sales_by_segment.merge(customer_counts, on=['LIFESTAGE', 'PREMIUM_CUSTOMER'])
segment_summary['AVG_SALES_PER_CUSTOMER'] = segment_summary['TOT_SALES'] / segment_summary['CUSTOMER_COUNT']
segment_summary = segment_summary.sort_values('AVG_SALES_PER_CUSTOMER', ascending=False)
segment_summary
```

Out[34]:

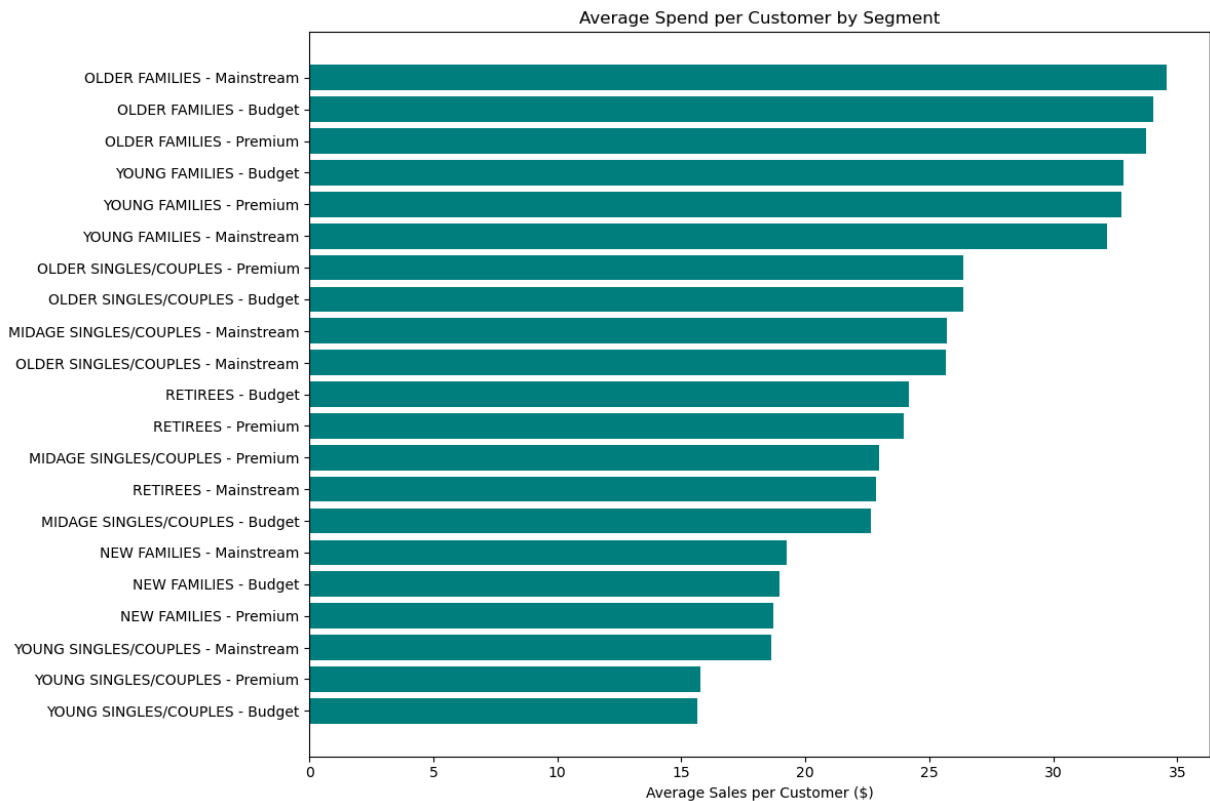
	LIFESTAGE	PREMIUM_CUSTOMER	TOT_SALES	CUSTOMER_COUNT	AVG_SALES_P
8	OLDER FAMILIES	Mainstream	96413.55	2788	
0	OLDER FAMILIES	Budget	156863.75	4611	
13	OLDER FAMILIES	Premium	75242.60	2231	
3	YOUNG FAMILIES	Budget	129717.95	3953	
12	YOUNG FAMILIES	Premium	78571.70	2398	
10	YOUNG FAMILIES	Mainstream	86338.25	2685	
6	OLDER SINGLES/COUPLES	Premium	123537.55	4682	
4	OLDER SINGLES/COUPLES	Budget	127833.60	4849	
11	MIDAGE SINGLES/COUPLES	Mainstream	84734.25	3298	
5	OLDER SINGLES/COUPLES	Mainstream	124648.50	4858	
7	RETIREEES	Budget	105916.30	4385	
9	RETIREEES	Premium	91296.65	3812	
15	MIDAGE SINGLES/COUPLES	Premium	54443.85	2369	
2	RETIREEES	Mainstream	145168.95	6358	
17	MIDAGE SINGLES/COUPLES	Budget	33345.70	1474	
19	NEW FAMILIES	Mainstream	15979.70	830	
18	NEW FAMILIES	Budget	20607.45	1087	
20	NEW FAMILIES	Premium	10760.80	575	
1	YOUNG SINGLES/COUPLES	Mainstream	147582.20	7917	
16	YOUNG SINGLES/COUPLES	Premium	39052.30	2480	
14	YOUNG SINGLES/COUPLES	Budget	57122.10	3647	



In [35]:

```
plt.figure(figsize=(12, 8))
segment_summary_sorted = segment_summary.sort_values('AVG_SALES_PER_CUSTOMER', asce
plt.barh(segment_summary_sorted['LIFESTAGE'] + ' - ' + segment_summary_sorted['PREM
        segment_summary_sorted['AVG_SALES_PER_CUSTOMER'], color='teal')
plt.xlabel('Average Sales per Customer ($)')
plt.title('Average Spend per Customer by Segment')
```

```
plt.tight_layout()
plt.savefig('avg_spend_per_customer.png', dpi=150)
plt.show()
```



```
In [36]: units_per_customer = merged_data.groupby(['LIFESTAGE', 'PREMIUM_CUSTOMER']).agg(
    TOTAL_UNITS=('PROD_QTY', 'sum'),
    CUSTOMERS=('LYLTY_CARD_NBR', 'nunique')
).reset_index()
units_per_customer['AVG_UNITS_PER_CUSTOMER'] = units_per_customer['TOTAL_UNITS'] /
units_per_customer.sort_values('AVG_UNITS_PER_CUSTOMER', ascending=False)
```

Out[36]:

	LIFESTAGE	PREMIUM_CUSTOMER	TOTAL_UNITS	CUSTOMERS	AVG_UNITS_PER_C
7	OLDER FAMILIES	Mainstream	25804	2788	
6	OLDER FAMILIES	Budget	41853	4611	
8	OLDER FAMILIES	Premium	20239	2231	
15	YOUNG FAMILIES	Budget	34482	3953	
17	YOUNG FAMILIES	Premium	20901	2398	
16	YOUNG FAMILIES	Mainstream	23194	2685	
9	OLDER SINGLES/COUPLES	Budget	32883	4849	
11	OLDER SINGLES/COUPLES	Premium	31695	4682	
10	OLDER SINGLES/COUPLES	Mainstream	32607	4858	
1	MIDAGE SINGLES/COUPLES	Mainstream	21213	3298	
12	RETIREEES	Budget	26932	4385	
14	RETIREEES	Premium	23266	3812	
2	MIDAGE SINGLES/COUPLES	Premium	14400	2369	
0	MIDAGE SINGLES/COUPLES	Budget	8883	1474	
13	RETIREEES	Mainstream	37677	6358	
4	NEW FAMILIES	Mainstream	4060	830	
3	NEW FAMILIES	Budget	5241	1087	
5	NEW FAMILIES	Premium	2769	575	
19	YOUNG SINGLES/COUPLES	Mainstream	36225	7917	
20	YOUNG SINGLES/COUPLES	Premium	10575	2480	
18	YOUNG SINGLES/COUPLES	Budget	15500	3647	

In [38]:

```
#pack size
pack_by_segment = merged_data.groupby(['LIFESTAGE', 'PREMIUM_CUSTOMER'])['PACK_SIZE']
pack_by_segment = pack_by_segment.sort_values('PACK_SIZE', ascending=False)
pack_by_segment
```

Out[38]:

	LIFESTAGE	PREMIUM_CUSTOMER	PACK_SIZE
19	YOUNG SINGLES/COUPLES	Mainstream	178.344249
1	MIDAGE SINGLES/COUPLES	Mainstream	177.898693
11	OLDER SINGLES/COUPLES	Premium	176.485568
12	RETIREES	Budget	176.395641
14	RETIREES	Premium	176.368421
4	NEW FAMILIES	Mainstream	175.629748
6	OLDER FAMILIES	Budget	175.546342
15	YOUNG FAMILIES	Budget	175.459720
9	OLDER SINGLES/COUPLES	Budget	175.334673
5	NEW FAMILIES	Premium	175.245296
13	RETIREES	Mainstream	175.213671
7	OLDER FAMILIES	Mainstream	175.175666
10	OLDER SINGLES/COUPLES	Mainstream	174.812145
3	NEW FAMILIES	Budget	174.766643
17	YOUNG FAMILIES	Premium	174.659032
2	MIDAGE SINGLES/COUPLES	Premium	174.585391
8	OLDER FAMILIES	Premium	174.524752
16	YOUNG FAMILIES	Mainstream	174.202059
0	MIDAGE SINGLES/COUPLES	Budget	174.182690
20	YOUNG SINGLES/COUPLES	Premium	174.002563
18	YOUNG SINGLES/COUPLES	Budget	173.378981

```
In [39]: most_common_pack = merged_data.groupby(['LIFESTAGE', 'PREMIUM_CUSTOMER'])['PACK_SIZE']
most_common_pack.columns = ['LIFESTAGE', 'PREMIUM_CUSTOMER', 'MOST_COMMON_PACK_SIZE']
most_common_pack
```


Out[39]:

	LIFESTAGE	PREMIUM_CUSTOMER	MOST_COMMON_PACK_SIZE
0	MIDAGE SINGLES/COUPLES	Budget	175
1	MIDAGE SINGLES/COUPLES	Mainstream	175
2	MIDAGE SINGLES/COUPLES	Premium	175
3	NEW FAMILIES	Budget	175
4	NEW FAMILIES	Mainstream	175
5	NEW FAMILIES	Premium	175
6	OLDER FAMILIES	Budget	175
7	OLDER FAMILIES	Mainstream	175
8	OLDER FAMILIES	Premium	175
9	OLDER SINGLES/COUPLES	Budget	175
10	OLDER SINGLES/COUPLES	Mainstream	175
11	OLDER SINGLES/COUPLES	Premium	175
12	RETIREEES	Budget	175
13	RETIREEES	Mainstream	175
14	RETIREEES	Premium	175
15	YOUNG FAMILIES	Budget	175
16	YOUNG FAMILIES	Mainstream	175
17	YOUNG FAMILIES	Premium	175
18	YOUNG SINGLES/COUPLES	Budget	175
19	YOUNG SINGLES/COUPLES	Mainstream	175
20	YOUNG SINGLES/COUPLES	Premium	175

```
In [40]: top_brand_by_segment = merged_data.groupby(['LIFESTAGE', 'PREMIUM_CUSTOMER', 'BRAND'])
top_brand_by_segment = top_brand_by_segment.sort_values('TOT_SALES', ascending=False)
top_brand_by_segment
```

Out[40]:

		BRAND	TOT_SALES
LIFESTAGE	PREMIUM_CUSTOMER		
MIDAGE SINGLES/COUPLES	Budget	Kettle	6736.2
	Mainstream	Kettle	20231.8
	Premium	Kettle	11390.2
NEW FAMILIES	Budget	Kettle	4731.6
	Mainstream	Kettle	3847.0
	Premium	Kettle	2267.6
OLDER FAMILIES	Budget	Kettle	32058.0
	Mainstream	Kettle	19420.2
	Premium	Kettle	14505.8
OLDER SINGLES/COUPLES	Budget	Kettle	29066.4
	Mainstream	Kettle	26852.8
	Premium	Kettle	27943.4
RETIREEES	Budget	Kettle	24340.0
	Mainstream	Kettle	31652.4
	Premium	Kettle	20922.4
YOUNG FAMILIES	Budget	Kettle	26369.6
	Mainstream	Kettle	17049.2
	Premium	Kettle	16614.2
YOUNG SINGLES/COUPLES	Budget	Kettle	11106.0
	Mainstream	Kettle	35423.6
	Premium	Kettle	7711.4

```
In [41]: target_segment = merged_data[merged_data['LIFESTAGE'] == 'OLDER FAMILIES']
other_segments = merged_data[merged_data['LIFESTAGE'] != 'OLDER FAMILIES']

target_brand_share = target_segment['BRAND'].value_counts(normalize=True)
other_brand_share = other_segments['BRAND'].value_counts(normalize=True)

brand_affinity = (target_brand_share / other_brand_share).sort_values(ascending=False)
brand_affinity
```

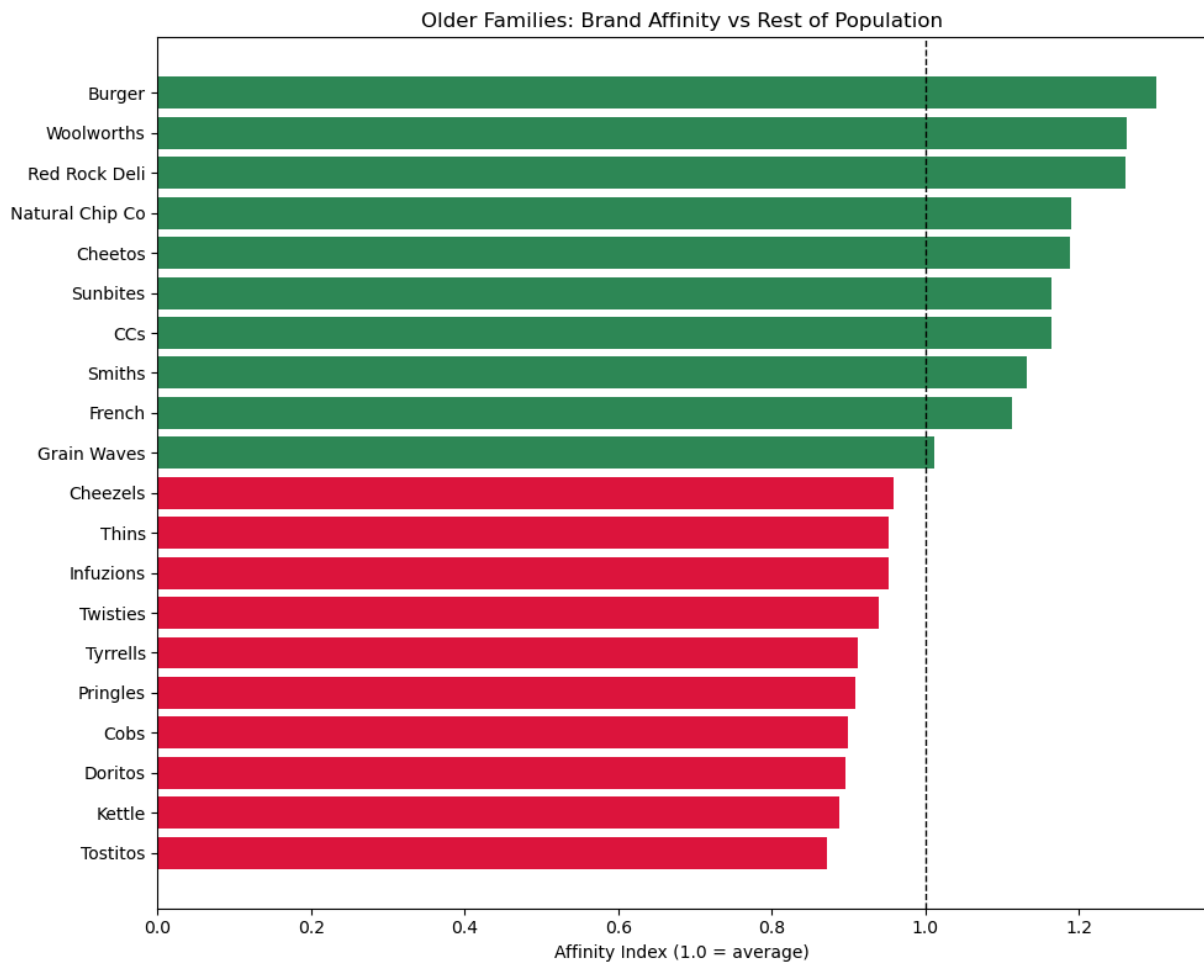
```
Out[41]: BRAND
Burger      1.301211
Woolworths  1.262207
Red Rock Deli  1.260126
Natural Chip Co  1.189018
Cheetos      1.187420
Sunbites     1.163689
CCs          1.163589
Smiths       1.131513
French       1.113032
Grain Waves  1.010767
Cheezels     0.957565
Thins        0.952433
Infuzions    0.951898
Twisties     0.939654
Tyrrells     0.912147
Pringles     0.908280
Cobs         0.898428
Doritos      0.896355
Kettle       0.888067
Tostitos     0.870818
Name: proportion, dtype: float64
```

```
In [42]: target_pack_share = target_segment['PACK_SIZE'].value_counts(normalize=True)
other_pack_share = other_segments['PACK_SIZE'].value_counts(normalize=True)

pack_affinity = (target_pack_share / other_pack_share).sort_values(ascending=False)
pack_affinity.head(10)
```

```
Out[42]: PACK_SIZE
180      1.340364
220      1.301211
200      1.273758
190      1.242202
160      1.221263
90       1.163689
70       1.160770
125      1.145883
165      1.093095
150      1.015365
Name: proportion, dtype: float64
```

```
In [43]: plt.figure(figsize=(10, 8))
brand_affinity_sorted = brand_affinity.sort_values()
colors = ['crimson' if x < 1 else 'seagreen' for x in brand_affinity_sorted]
plt.barh(brand_affinity_sorted.index, brand_affinity_sorted.values, color=colors)
plt.axvline(x=1, color='black', linestyle='--', linewidth=1)
plt.xlabel('Affinity Index (1.0 = average)')
plt.title('Older Families: Brand Affinity vs Rest of Population')
plt.tight_layout()
plt.savefig('older_families_brand_affinity.png', dpi=150)
plt.show()
```



```
In [44]: target_segment2 = merged_data[merged_data['LIFESTAGE'] == 'YOUNG FAMILIES']
other_segments2 = merged_data[merged_data['LIFESTAGE'] != 'YOUNG FAMILIES']

target_brand_share2 = target_segment2['BRAND'].value_counts(normalize=True)
other_brand_share2 = other_segments2['BRAND'].value_counts(normalize=True)

brand_affinity2 = (target_brand_share2 / other_brand_share2).sort_values(ascending=
brand_affinity2
```

```
Out[44]: BRAND
Sunbites      1.258531
CCs           1.252050
French        1.242038
Red Rock Deli 1.208064
Cheetos       1.178497
Burger        1.174132
Woolworths    1.169992
Natural Chip Co 1.141015
Smiths        1.101966
Cheezels      1.024764
Grain Waves   0.953959
Infuzions     0.941227
Thins         0.936483
Cobs          0.935431
Tostitos      0.933508
Tyrrells      0.932593
Doritos       0.929822
Pringles      0.916751
Kettle        0.913151
Twisties      0.894263
Name: proportion, dtype: float64
```

```
In [45]: sales_by_segment.to_csv('sales_by_segment.csv', index=False)
segment_summary.to_csv('segment_summary_avg_spend_units.csv', index=False)
pack_by_segment.to_csv('pack_size_by_segment.csv', index=False)
brand_affinity.to_csv('older_families_brand_affinity.csv', header=['affinity_index'])
brand_affinity2.to_csv('young_families_brand_affinity.csv', header=['affinity_index'])
print("All analysis files saved!")
```

All analysis files saved!

Summary of Findings

Total Sales Leaders: Older Families/Budget, Young Singles-Couples/Mainstream, and Retirees/Mainstream generate the highest total sales — but this is driven by large customer bases, not high spend per customer.

Highest total sales: Older Families/Budget, Young Singles-Couples/Mainstream, Retirees/Mainstream — driven mainly by having large customer bases, not necessarily high per-customer value.

Highest value customers: Older and Young Families spend the most per customer (~32–35) and buy the most units (8.6–9.3 per customer) — nearly double that of Young Singles (15–19, ~4.3–4.6 units).

Pack size: Not a meaningful differentiator between segments — preference is fairly consistent across all groups (~173–178g average).

Brand preference: A meaningful differentiator — Families (both Older and Young) consistently favor value/everyday brands: Woolworths, Red Rock Deli, Sunbites, CCs, Natural

Chip Co, Cheetos, Burger Rings — and consistently avoid premium/mainstream leaders like Kettle, Pringles, Doritos, Tostitos.

Recommendation

Target marketing and promotions toward Families (Older and Young) as the highest-value segment on a per-customer basis, specifically promoting value-oriented, everyday snack brands (Woolworths, Red Rock Deli, Sunbites, CCs) rather than premium lines, since that's where their actual purchasing behavior already sits. Meanwhile, Young Singles/Couples remain a high-volume segment worth retaining, but with lower expected return per customer, so may warrant lower-cost engagement (e.g. digital/loyalty app promotions) rather than the same investment level as Families.

```
In [46]: import os
         for f in os.listdir():
             if f.endswith('.csv') or f.endswith('.png'):
                 print(f)
```

```
avg_spend_per_customer.png
EdTech.csv
heartdisease.csv
housing - housing.csv
older_families_brand_affinity.csv
older_families_brand_affinity.png
pack_size_by_segment.csv
QVI_merged_data.csv
QVI_purchase_behaviour.csv
QVI_transaction_data_clean.csv
sales_by_segment.csv
sales_by_segment.png
segment_summary_avg_spend_units.csv
young_families_brand_affinity.csv
```

```
In [ ]:
```